

2

构造/析构/赋值运算

Constructors, Destructors, and Assignment Operators

几乎你写的每一个 `class` 都会有一或多个构造函数、一个析构函数、一个 *copy assignment* 操作符。这些很难让你特别兴奋，毕竟它们是你的基本谋生工具，控制着基础操作，像是产出新对象并确保它被初始化、摆脱旧对象并确保它被适当清理、以及赋予对象新值。如果这些函数犯错，会导致深远且令人不愉快的后果，遍及你的整个 `classes`。所以确保它们行为正确是生死攸关的大事。本章提供的引导可让你把这些函数良好地集结在一起，形成 `classes` 的脊柱。

条款 05：了解 C++ 默默编写并调用哪些函数

Know what functions C++ silently writes and calls.

什么时候 `empty class`（空类）不再是个 `empty class` 呢？当 C++ 处理过它之后。是的，如果你自己没声明，编译器就会为它声明（编译器版本的）一个 *copy* 构造函数、一个 *copy assignment* 操作符和一个析构函数。此外如果你没有声明任何构造函数，编译器也会为你声明一个 *default* 构造函数。所有这些函数都是 `public` 且 `inline`（见条款 30）。因此，如果你写下：

```
class Empty { };
```

这就好像你写下这样的代码：

```
class Empty {
public:
    Empty() { ... }                // default 构造函数
    Empty(const Empty& rhs) { ... } // copy 构造函数
    ~Empty() { ... }              // 析构函数，是否该是
                                // virtual 见稍后说明.
    Empty& operator=(const Empty& rhs) { ... } // copy assignment 操作符.
};
```

惟有当这些函数被需要（被调用），它们才会被编译器创建出来。程序中需要它们是很平常的事。下面代码造成上述每一个函数被编译器产出：

```
Empty e1;                //default 构造函数
                          //析构函数
Empty e2(e1);            //copy 构造函数
e2 = e1;                  //copy assignment 操作符
```

好，我们知道了，编译器为你写函数，但这些函数做了什么呢？唔，*default* 构造函数和析构函数主要是给编译器一个地方用来放置“藏身幕后”的代码，像是调用 *base classes* 和 *non-static* 成员变量的构造函数和析构函数。注意，编译器产出的析构函数是个 *non-virtual*（见条款 7），除非这个 *class* 的 *base class* 自身声明有 *virtual* 析构函数（这种情况下这个函数的虚属性；*virtualness*；主要来自 *base class*）。

至于 *copy* 构造函数和 *copy assignment* 操作符，编译器创建的版本只是单纯地将来源对象的每一个 *non-static* 成员变量拷贝到目标对象。考虑一个 *NamedObject* *template*，它允许你将一个个名称和类型为 *T* 的对象产生关联：

```
template<typename T>
class NamedObject {
public:
    NamedObject(const char* name, const T& value);
    NamedObject(const std::string& name, const T& value);
    ...
private:
    std::string nameValue;
    T objectValue;
};
```

由于其中声明了一个构造函数，编译器于是不再为它创建 *default* 构造函数。这很重要，意味如果你用心设计一个 *class*，其构造函数要求实参，你就无须担心编译器会毫无挂虑地为你添加一个无实参构造函数（即 *default* 构造函数）而遮盖掉你的版本。

NamedObject 既没有声明 *copy* 构造函数，也没有声明 *copy assignment* 操作符，所以编译器会为它创建那些函数（如果它们被调用的话）。现在，看看 *copy* 构造函数的用法：

```
NamedObject<int> no1("Smallest Prime Number", 2);
NamedObject<int> no2(no1);                //调用 copy 构造函数
```

编译器生成的 *copy* 构造函数必须以 `no1.nameValue` 和 `no1.objectValue` 为初值设定 `no2.nameValue` 和 `no2.objectValue`。两者之中，`nameValue` 的类型是 `string`，而标准 `string` 有个 *copy* 构造函数，所以 `no2.nameValue` 的初始化方式是调用 `string` 的 *copy* 构造函数并以 `no1.nameValue` 为实参。另一个成员 `NamedObject<int>::objectValue` 的类型是 `int`（因为对此 *template* 具现体而言 `T` 是 `int`），那是个内置类型，所以 `no2.objectValue` 会以“拷贝 `no1.objectValue` 内的每一个 bits”来完成初始化。

编译器为 `NamedObject<int>` 所生的 *copy assignment* 操作符，其行为基本上与 *copy* 构造函数如出一辙，但一般而言只有当生出的代码合法且有适当机会证明它有意义（见下页），其表现才会如我先前所说。万一两个条件有一个不符合，编译器会拒绝为 `class` 生出 `operator=`。

举个例子，假设 `NamedObject` 定义如下，其中 `nameValue` 是个 *reference to* `string`，`objectValue` 是个 `const T`：

```
template<class T>
class NamedObject {
public:
    //以下构造函数如今不再接受一个 const 名称，因为 nameValue
    //如今是个 reference-to-non-const string。先前那个 char* 构造函数
    //已经过去了，因为必须有个 string 可供指涉。
    NamedObject(std::string& name, const T& value);
    ...
private:
    std::string& nameValue; //这如今是个 reference
    const T objectValue;   //这如今是个 const
};
```

现在考虑下面会发生什么事：

```
std::string newDog("Persephone");
std::string oldDog("Satch");
NamedObject<int> p(newDog, 2); //当初撰写至此，我们的狗 Persephone
                               //即将度过其第二个生日。
NamedObject<int> s(oldDog, 36); //我小时候养的狗 Satch 则是 36 岁，
                               //-- 如果她还活着。
p = s;                          //现在 p 的成员变量该发生什么事？
```

赋值之前，不论 `p.nameValue` 和 `s.nameValue` 都指向 `string` 对象（当然不是同一个）。赋值动作该如何影响 `p.nameValue` 呢？赋值之后 `p.nameValue` 应该

指向 `s.nameValue` 所指的那个 `string` 吗? 也就是说 `reference` 自身可被改动吗? 如果是, 那可就开辟了新天地, 因为 C++ 并不允许“让 `reference` 改指向不同对象”。换一个想法, `p.nameValue` 所指的那个 `string` 对象该被修改, 进而影响“持有 `pointers` 或 `references` 而且指向该 `string`”的其他对象吗? 也就是对象不被直接牵扯到赋值操作内? 编译器生成的 `copy assignment` 操作符究竟该怎么做呢?

面对这个难题, C++ 的响应是拒绝编译那一行赋值动作。如果你打算在一个“内含 `reference` 成员”的 `class` 内支持赋值操作 (`assignment`), 你必须自己定义 `copy assignment` 操作符。面对“内含 `const` 成员” (如本例之 `objectValue`) 的 `classes`, 编译器的反应也一样。更改 `const` 成员是不合法的, 所以编译器不知道如何在它自己生成的赋值函数内面对它们。最后还有一种情况: 如果某个 `base classes` 将 `copy assignment` 操作符声明为 `private`, 编译器将拒绝为其 `derived classes` 生成一个 `copy assignment` 操作符。毕竟编译器为 `derived classes` 所生的 `copy assignment` 操作符想象中可以处理 `base class` 成分 (见条款 12), 但它们当然无法调用 `derived class` 无权调用的成员函数。编译器两手一摊, 无能为力。

请记住

- 编译器可以暗自为 `class` 创建 `default` 构造函数、`copy` 构造函数、`copy assignment` 操作符, 以及析构函数。

条款 06: 若不想使用编译器自动生成的函数, 就该明确拒绝

Explicitly disallow the use of compiler-generated functions you do not want.

地产中介商卖的是房子, 一个中介软件系统自然而然想必有个 `class` 用来描述待售房屋:

```
class HomeForSale { ... };
```

每一位真正的地产中介商都会说, 任何一笔资产都是天上地下独一无二, 没有两笔完全相像。因此我们也认为, 为 `HomeForSale` 对象做一份副本有点没道理。你怎么可以复制某些先天独一无二的东西呢? 因此, 你应该乐意看到 `HomeForSale` 的对象拷贝动作以失败收场:

```
HomeForSale h1;  
HomeForSale h2;  
HomeForSale h3(h1);           //企图拷贝 h1 — 不该通过编译  
h1 = h2;                       //企图拷贝 h2 — 也不该通过编译
```